

PROJECT DOCUMENTATION

DOCK

Porting Manual

모바일 더치페이 / 정산 서비스

프로젝트명	DOCK (더치페이/정산 서비스)
문서 유형	Porting Manual (포팅 매뉴얼)
프론트엔드	Expo + React Native (Android APK)
백엔드	Spring Boot 3.5 Microservices
인프라	AWS EC2 / Docker Compose / Jenkins / RDS / Kafka / S3

1. 프로젝트 개요

DOCK은 모바일 기반 더치페이/정산 서비스입니다. 프론트엔드는 Expo 기반 React Native 앱으로 구성되어 있으며, 백엔드는 Spring Boot 마이크로 서비스 구조로 구성되어 있습니다.

주요 기능

- 카카오 소셜 로그인
- 계좌 등록 및 1원 인증
- 모임방 생성 / 초대 링크 공유 / 참여
- 결제 내역 등록 및 정산
- 월별 소비 리포트
- FCM 알림
- 프로필 이미지 S3 업로드

2. 시스템 구성

2.1 프론트엔드

경로	DOCK-FE
프레임워크	Expo + React Native
배포 방식	EAS Build를 통한 Android APK 배포

2.2 백엔드

경로: DOCK-BE | 구성 서비스:

- gateway-service
- discovery-service
- core-service
- pay-service
- insight-service

2.3 인프라

- AWS EC2
- Docker Compose
- AWS RDS MySQL
- Redis
- Kafka
- AWS S3
- Firebase Cloud Messaging
- Grafana / Prometheus
- Jenkins
- Nginx

3. 기술 스택 및 버전

3.1 Frontend

Node.js	>= 22.11.0
Expo	^54.0.33

React	19.1.0
React Native	0.81.5
TypeScript	^5.8.3
EAS CLI	>= 18.4.0
Firebase App	^23.8.8
Firebase Messaging	^23.8.8
Notifee	^9.1.8

3.2 Backend

Java	17
Spring Boot	3.5.11
Spring Cloud	2025.0.1
빌드 도구	Gradle
주요 의존성	Spring Data JPA, Spring Data Redis, Spring Kafka
서비스 등록	Spring Cloud Netflix Eureka
서비스 간 통신	Spring Cloud OpenFeign
API 문서	Springdoc OpenAPI
메트릭	Micrometer Prometheus Registry

3.3 Infra / DevOps

컨테이너	Docker / Docker Compose
CI/CD	Jenkins Pipeline
이미지 레지스트리	Docker Hub
서버	AWS EC2
데이터베이스	AWS RDS (MySQL)
파일 스토리지	AWS S3
푸시 알림	Firebase FCM
모니터링	Grafana / Prometheus

4. 레포지토리 구조

```
S14P21C102
├─ DOCK-BE
│  └─ core-service
│  └─ discovery-service
│  └─ gateway-service
│  └─ insight-service
│  └─ pay-service
├─ DOCK-FE
├─ docs
├─ tools
├─ docker-compose.local.yml
├─ docker-compose.prod.yml
├─ Jenkinsfile
└─ PORTING_MANUAL.md
```

5. 사전 설치 항목

5.1 공통

- Git
- Docker
- Docker Compose

5.2 백엔드 개발 환경

- JDK 17
- IntelliJ IDEA 또는 Gradle 실행 환경

5.3 프론트엔드 개발 환경

- Node.js 22 이상
- npm
- Android Studio
- Android SDK
- Expo CLI 실행 환경
- EAS CLI

EAS CLI 설치:

```
npm install -g eas-cli
```

6. 환경 변수 설정

6.1 공통 .env

이 프로젝트는 레포지토리 루트의 .env 파일을 기준으로 주요 설정을 읽습니다.

- 프론트: DOCK-FE/app.config.js에서 루트 .env를 읽음
- 운영 백엔드: docker-compose.prod.yml의 env_file로 사용

6.2 필수 환경 변수

민감 값은 실제 운영 값으로 교체해야 합니다.

```
SPRING_PROFILES_ACTIVE=prod

DB_HOST=
DB_PORT=3306
DB_USERNAME=
DB_PASSWORD=
```

```

CORE_DB_NAME=duckchi_core
PAY_DB_NAME=duckchi_pay
INSIGHT_DB_NAME=duckchi_insight

REDIS_HOST=redis
REDIS_PORT=6379

EUREKA_SERVER_URL=http://discovery-service:8761/eureka
EUREKA_HOST=discovery-service
EUREKA_PORT=8761

GATEWAY_PORT=8080
CORE_PORT=8081
PAY_PORT=8082
INSIGHT_PORT=8083
DISCOVERY_PORT=8761

JWT_SECRET=
JWT_ACCESS_EXPIRATION=3600000
JWT_REFRESH_EXPIRATION=1209600000

KAKAO_CLIENT_ID=
KAKAO_REDIRECT_URI=https://도메인/api/v1/auth/oauth/login

FINANCE_API_KEY=
PAY_PASSWORD_PEPPER=

OCR_INVOKE_URL=
OCR_SECRET_KEY=

FIREBASE_PROD_ACCOUNT_PATH=/home/ubuntu/duckchi/secrets/service-account.json

AWS_ACCESS_KEY=
AWS_SECRET_KEY=
AWS_REGION=ap-northeast-2
AWS_S3_BUCKET=
AWS_S3_PUBLIC_BASE_URL=
AWS_S3_PROFILE_PREFIX=profiles
AWS_S3_PREIGNED_URL_EXPIRATION_MINUTES=10

GRAFANA_ADMIN_PASSWORD=

KAFKA_BOOTSTRAP_SERVERS=kafka:9092
SPRING_KAFKA_BOOTSTRAP_SERVERS=kafka:9092
SPRING_KAFKA_CONSUMER_GROUP_ID=insight-service-group
SPRING_KAFKA_CONSUMER_AUTO_OFFSET_RESET=earliest
SPRING_KAFKA_CONSUMER_KEY_DESERIALIZER=org.apache.kafka.common.serialization.StringDeserializer
SPRING_KAFKA_CONSUMER_VALUE_DESERIALIZER=org.springframework.kafka.support.serializer.JsonDeserializer
SPRING_KAFKA_CONSUMER_PROPERTIES_SPRING_JSON_TRUSTED_PACKAGES=com.duckchi.insight.*
SPRING_KAFKA_LISTENER_ACK_MODE>manual_immediate

API_BASE_URL=https://도메인
INVITE_LINK_BASE_URL=https://도메인/invite
INVITE_LINK_HOST=도메인

```

6.3 주의 사항

- KAFKA_BOOTSTRAP_SERVERS는 반드시 kafka:9092 형태로 설정
- API_BASE_URL과 KAKAO_REDIRECT_URI는 HTTPS 운영 도메인 기준으로 설정
- 프론트는 루트 .env를 읽으므로 DOCK-FE/.env가 아니라 레포지토리 루트 .env를 기준으로 관리

7. 외부 서비스 준비

7.1 MySQL (RDS)

운영에서는 AWS RDS MySQL을 사용합니다.

- DB 생성: duckchi_core, duckchi_pay, duckchi_insight
- 보안 그룹에서 EC2 및 개발자 IP의 3306 접근 허용 필요
- 개발자 로컬에서 Workbench 접속 시 현재 공인 IP를 /32로 등록 필요

7.2 Redis

- 운영: docker-compose.prod.yml에서 Redis 컨테이너 사용
- 로컬: docker-compose.local.yml에서 Redis Stack 사용

7.3 Kafka

- 로컬: docker-compose.local.yml
- 운영: docker-compose.prod.yml 내 kafka 서비스 사용
- 운영 환경 변수: KAFKA_BOOTSTRAP_SERVERS=kafka:9092

7.4 Kakao Developers

- Kakao Login 활성화
- Redirect URI 등록: <https://도메인/api/v1/auth/oauth/login>
- 필요한 동의항목 설정

7.5 Firebase

- Android: D0CK-FE/google-services.json
- 운영 서버: Firebase Admin SDK 서비스 계정 JSON
- 예시 경로: /home/ubuntu/duckchi/secrets/service-account.json

7.6 AWS S3

- 버킷 생성
- 액세스 키 / 시크릿 키 발급
- 퍼블릭 베이스 URL 설정

7.7 OCR

- OCR API invoke URL 및 secret key 필요

8. 로컬 실행 방법

8.1 인프라 실행

레포지토리 루트에서 실행 (실행 대상: Kafka, Redis)

```
docker compose -f docker-compose.local.yml up -d
```

8.2 백엔드 실행

권장 실행 순서:

1. discovery-service
2. gateway-service
3. core-service
4. pay-service
5. insight-service

```
cd DOCK-BE/discovery-service
./gradlew bootRun
```

8.3 프론트 실행

```
cd DOCK-FE
npm install
npx expo start -c
```

안드로이드 네이티브 빌드:

```
npx expo run:android
```

8.4 로컬 포트

서비스	포트
Gateway	8080
Core	8081
Pay	8082
Insight	8083
Discovery (Eureka)	8761
Redis	6379
Redis Stack UI	8001
Kafka	9092

9. 운영 배포 방법

9.1 운영 서버 구조

호스트	AWS EC2
배포 디렉터리	/home/ubuntu/duckchi
Compose 파일	docker-compose.prod.yml
배포 방식	Jenkins가 Docker 이미지 빌드/푸시 후 EC2에서 compose 재기동

9.2 Docker Compose 운영 실행

```
cd /home/ubuntu/duckchi  
REGISTRY_NAMESPACE=ryubyeongsun1 IMAGE_TAG=latest docker compose -f docker-compose.prod.yml up -d --remove-orphans
```

9.3 운영 Compose 서비스 목록

- kafka
- redis
- discovery-service
- gateway-service
- core-service
- pay-service
- insight-service
- prometheus
- grafana
- node-exporter
- cadvisor

Gateway만 외부 포트(127.0.0.1:8080:8080)에 노출되며, 실제 외부 요청은 Nginx를 통해 HTTPS로 들어와 내부 Gateway 8080으로 프록시됩니다.

10. Jenkins CI/CD

Jenkinsfile 위치	루트 Jenkinsfile
Jenkins 접속	http://3.39.255.72:9090 또는 http://j14c102.p.ssafy.io:9090
사용 잡	duckchi-ci, duckchi-cd-release
필수 Credentials	Docker Hub 계정, EC2 SSH 키

10.1 파이프라인 동작 순서

1. 소스 체크아웃
2. 백엔드 서비스 구조 검증
3. 백엔드 Gradle 빌드
4. 프론트 Lint / Test (CI 잡일 때)
5. Dockerfile 존재 검증
6. 실행 가능한 JAR 추출
7. Docker 이미지 빌드 및 Docker Hub 푸시
8. EC2 SSH 접속 후 docker compose 재배포

11. 프론트 APK 빌드 및 배포

11.1 EAS Build 프로필

DOCK-FE/eas.json 기준 사용 프로필: development, preview, production

배포 테스트용 APK 빌드:

```
cd DOCK-FE
npx eas-cli build --platform android --profile preview --clear-cache
```

11.2 필수 파일

- DOCK-FE/google-services.json

11.3 APK 재배포가 필요한 경우

- 프론트 코드 변경
- API_BASE_URL 변경
- KAKAO_REDIRECT_URI 변경
- Firebase 설정 변경
- 딥링크 / App Link 설정 변경

백엔드만 수정된 경우에는 APK 재배포 없이 서버 CD만으로 반영됩니다.

12. 초대 링크 (App Link) 설정

Android Package	com.ssafy.duckduck
iOS Bundle ID	com.ssafy.duckduck
Scheme	duckchi
초대 링크 형식	https://도메인/invite/{token}
assetlinks.json 경로	https://도메인/.well-known/assetlinks.json

12.1 설정 조건

- assetlinks.json에 현재 테스트/배포 APK의 SHA-256 포함
- debug 빌드와 EAS 빌드의 SHA가 다를 수 있으므로 각각 확인 필요
- AndroidManifest / Expo intentFilters에 /invite 경로 포함
- 앱 삭제 후 재설치 필요 (Android가 App Link 검증 결과를 캐시하기 때문)

13. 모니터링 구성

13.1 Spring Boot 메트릭 엔드포인트

- /actuator/health
- /actuator/info
- /actuator/metrics
- /actuator/prometheus

13.2 Prometheus 수집 대상 (운영)

- core-service:8081/actuator/prometheus
- gateway-service:8080/actuator/prometheus
- pay-service:8082/actuator/prometheus
- insight-service:8083/actuator/prometheus
- discovery-service:8761/actuator/prometheus

13.3 Grafana

접속 URL	https://j14c102.p.ssafy.io/grafana/login
권장 대시보드	Spring Boot 3.x Statistics

14. 데이터베이스 초기화

필요 시 RDS 또는 로컬 MySQL에서 다음 DB를 재생성합니다.

- duckchi_core
- duckchi_pay
- duckchi_insight

⚠ JPA ddl-auto: update는 최신 DDL을 완전히 재구성하지 못할 수 있습니다. 스키마가 크게 바뀌었을 경우 최신 SQL을 기준으로 DB를 drop/create 후 재가동하는 것이 안전합니다.

15. 자주 발생하는 이슈

15.1 MySQL Workbench에서 RDS 접속 불가

원인

현재 공인 IP가 RDS 보안 그룹에 등록되지 않음

해결

RDS 보안 그룹 인바운드에 MySQL/Aurora, 3306, 내 IP/32 추가

15.2 카카오 로그인 network error

원인

API_BASE_URL / KAKAO_REDIRECT_URI 불일치, EAS preview 환경변수가 HTTP로 남아있음, Kakao Developers Redirect URI 불일치

해결

Kakao Developers 콘솔 Redirect URI와 환경변수 일치 여부 확인, HTTPS 설정 확인

15.3 계좌 등록 실패

원인

금융망 전송시각 생성 시 서버/컨테이너 타임존 불일치 (운영 컨테이너 UTC vs 로컬 KST)

해결

금융 헤더 생성 시 Asia/Seoul 기준으로 날짜/시간 생성. `LocalDateTime.now()` 대신 명시적 `ZoneId.of("Asia/Seoul")` 사용

15.4 초대 링크가 브라우저 404로 열림

원인

App Link 검증 실패, `assetlinks.json` SHA 누락, 앱 재설치 미실행

해결

`assetlinks.json`에 올바른 SHA-256 등록 후 앱 삭제 및 재설치

15.5 Expo 네이티브 모듈 오류

원인

Notiffee native module not found, react-native-svg 모듈 누락 등

해결

`npm install` → `npx expo run:android`, 필요시 `npx expo start -c`

16. 제출용 요약

본 프로젝트는 Expo 기반 모바일 앱과 Spring Boot 마이크로서비스 백엔드로 구성되어 있으며, 운영 환경에서는 EC2 + Docker Compose + Jenkins + RDS + Redis + Kafka + S3 + Firebase + Grafana/Prometheus 기반으로 배포됩니다.

포팅 시 핵심 체크리스트

- ✓ 루트 .env 파일 설정 (DB, Redis, Kafka, JWT, Kakao, Firebase, S3, OCR)
- ✓ Firebase 설정 파일 배치 (google-services.json, service-account.json)
- ✓ Kakao Redirect URI 설정 (Kakao Developers 콘솔 및 환경변수)
- ✓ RDS 보안 그룹 설정 (EC2 및 개발자 IP 3306 허용)
- ✓ Docker Hub / Jenkins Credential 등록
- ✓ Kafka 운영 설정 (kafka:9092)
- ✓ App Link용 assetlinks.json 설정 및 Nginx 서빙

운영 배포는 release 브랜치 기준 Jenkins CD 또는 EC2에서의 Compose 재기동으로 수행합니다.